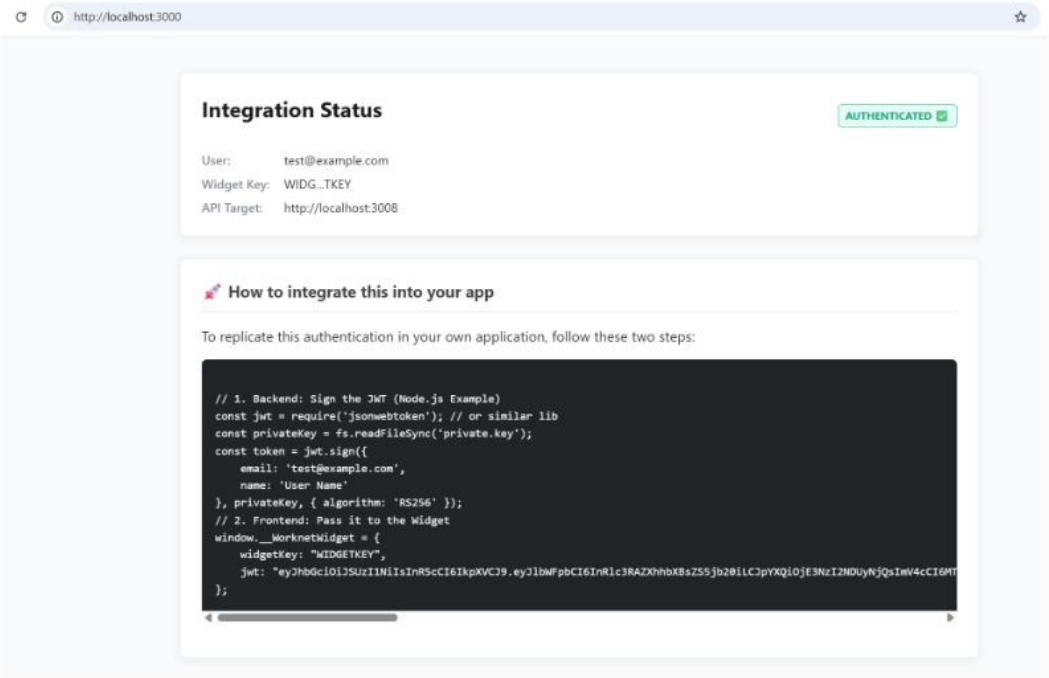
JWT Integration Test Server

Test Server Setup

Ran this command: node -e "const {generateKeyPairSync}=require('crypto');const k=generateKeyPairSync('rsa', {modulusLength:2048});require('fs').writeFileSync('private.key',k.privateKey.export({type:'pkcs8',format:'pem'}))"

- Generates an RSA key pair
- exports the private key in pkcs8/pem format
- saves it in my file with the server.js

run : node server.js



The test server ran successfully and authenticated.

Key Definitions

Encryption: scrambling data so no one can read it - taking plaintxt and making it unreadable (ciphertxt) using algorithms and a secret key

Signing: proving data is genuine and hasn't been tampered with (integrity)

Hashing: process of transforming any size of input data to a fixed length using a mathematical algorithm.

- Ensures integrity and security
- is one way- irreversible
- same output always produces same hash

Term	What is it	Role in JWT
SHA-256	Hashing Algorithm	creates a fingerprint of your data
RSA	Asymmetric Key Algorithm	signs/verifies that fingerprint
RS256	SHA-256 + RSA combined	the full signing process
Private Key	your secret key	signs the token
Public Key	shared with Worknet	verifies the token

JWT string: the JWT strings look like gibberish but consist of 3 pieces of structed data joined by dots.

- header.payload.signature
- All encoded in base64URL

JWT - JSON Web Token

Worknet needs to know who you are before we can use the widget.

- Instead of username/pass, it uses a JWT token
- token contains info about you (email, name, ect) and is cryptographically signed.
- Worknet can then verify that it wasn't faked or tampered with

Step 1: Generate Key Pair

create 2 mathematically linked keys

- private key: sits on server, never shared with anyone
- public key: uploaded to worknet admin dashboard

Private key creates signatures, public key verifies them and is impossible to reverse engineer the private key from public key

Step 2: Your server signs a JWT

when someone hits the localhost 3000, your server:

- builds a payload {email: "test@example.com}
- runs it through SHA-256 to get fingerprint
- locks fingerprint with private key
- produces the JWT: header.payload.signature

Step 3: JWT travels to worknet

page loads, Worknet widget picks up JWT from window.__WorknetWidget and sends it to the Worknet servers

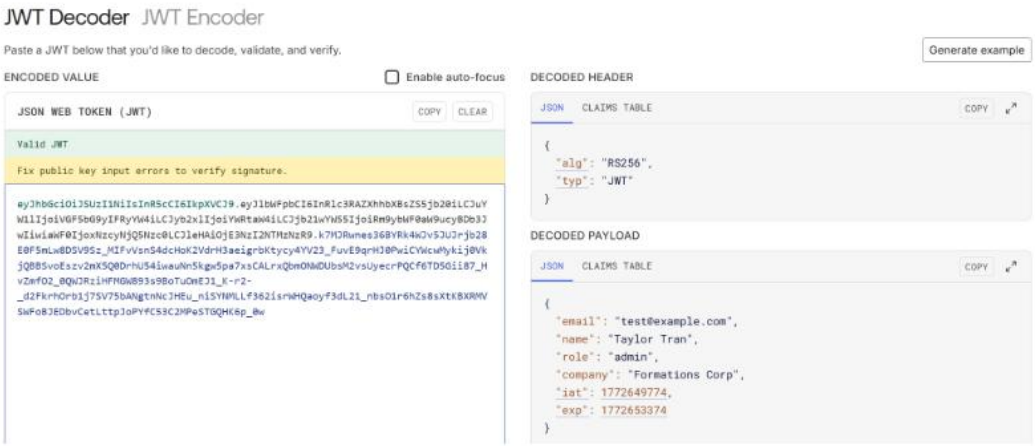
Step 4: Worknet verifies it

Worknet takes the JWT and:

- looks up your public key(registered in the dashboard)
- uses it to unlock the signature and recover fingerprint
- independently runs SHA-256 on the header + payload
- compares 2 finger prints

If it matches, you are who you say you are. if not, rejected.

Adding extra fields:



During local testing, JWT's are readable by design because **JWTS are NOT ENCRYPTED, they are SIGNED**. In real environments like production, the integration will run over HTTPS which automatically encrypts the connection, preventing the JWT from being intercepted while in transit. The payload itself is always going to be readable if decoded, so sensative data should not be stored in the JWT payload.

Sample JWT notes

```
// Default configuration
const DEFAULT_PRIVATE_KEY_PATH = join(__dirname, 'private.key');
const DEFAULT_WIDGET_KEY = "WIDGETKEY";
const DEFAULT_API_BASE_URL = 'http://localhost:3008';
const DEFAULT_CHAT_BASE_URL = 'http://localhost:5173';

// default settings, if we don't pass anything in the URL, these values get used.
// Replace WIDGETKEY with real worknet one
// Replace URLS with real endpoints
```

Testing with Worknet Widget Demo

```
  Buffer.from(input).toString('base64').replace(/\+/g, '-').replace(/\//g, '_').replace(

// Sign RS256 JWT
function signJwt(payload, privateKey) {
  const header = { alg: 'RS256', typ: 'JWT' };
  const now = Math.floor(Date.now() / 1000);
  const claims = { ...payload, iat: now, exp: now + 3600 };

  const encodedHeader = b64url(JSON.stringify(header));
  const encodedPayload = b64url(JSON.stringify(claims));
  const toSign = `${encodedHeader}.${encodedPayload}`;

  const signer = createSign('RSA-SHA256');
  signer.update(toSign);
  signer.end();

  const signature = signer.sign(privateKey, 'base64');
  const encodedSig = signature.replace(/\+/g, '-').replace(/\//g, '_').replace(/=/g, '')
  return `${encodedHeader}.${encodedPayload}.${encodedSig}`;
}

// Mask widget key for display
function maskKey(key) {
  if (!key || key.length < 8) {
    return key;
  }
  return `${key.substring(0, 4)}...${key.substring(key.length - 4)}`;
}

// Generate HTML page
function generateHTML(jwtToken, widgetKey, apiBaseUrl, chatBaseUrl, userEmail) {
  // We prepare a code snippet string to display on the screen
  const exampleCode = `
// 1. Backend: Sign the JWT (Node.js Example)
const jwt = require('jsonwebtoken'); // or similar lib
const privateKey = fs.readFileSync('private.key');
const token = jwt.sign({
    email: '${userEmail}',
    sub: 'user-123'
    name: 'User Name'
}, privateKey, { algorithm: 'RS256' });

// 2. Frontend: Pass it to the Widget
window.__WorknetWidget = {
    widgetKey: "${widgetKey}",
```